Design and Implementation of a Mechanical 7-Segment Display Clock Using Arduino and Servo Motors

Abstract

This project presents the design and implementation of a mechanical 7-segment display clock controlled by an Arduino Uno and actuated using 28 micro servo motors. The clock displays real time through four large mechanical digits and two center dots, where each segment is physically rotated into either a visible or hidden position. The system uses a DS1302 real-time clock module for timekeeping and two PCA9685 PWM driver boards to control the large number of servos efficiently through the I2C interface.

To improve practicality and continuous operation, the clock is powered by a regulated 5V, 5A wall power supply instead of a battery-based system. The DS1302 module also includes a backup battery, allowing it to continue keeping time even when the main power is disconnected. As a result, when the system is plugged in again, it displays the correct current time without requiring manual reset. The project also includes a structured 3D-printed frame with dedicated mounting locations for the servos and display parts, resulting in a stable and organized final prototype. Overall, the project demonstrates the integration of embedded systems, real-time control, servo actuation, wiring organization, and mechanical assembly in one functional electromechanical device.

1. Introduction

Most digital clocks use LEDs or LCDs to display time, but a mechanical display creates a more physical and visually engaging effect by moving real parts. In this project, each digit is made from seven mechanical segments, and each segment is controlled by an individual servo motor. By rotating the segments between visible and hidden positions, the system forms the required hour and minute digits in a 7-segment style.

The project was built using an Arduino Uno as the main controller, a DS1302 RTC module for accurate timekeeping, and two PCA9685 PWM driver boards to expand the number of available servo outputs. A total of 28 servo motors are used to actuate the four digits, while the two center dots complete the clock layout. To create a more practical and reliable build, the system was powered directly from a household wall outlet using a regulated 5V supply, and the mechanical structure was assembled into a dedicated 3D-printed frame. The final result is a complete mechanical clock that combines electronics, programming, and fabrication in one integrated system.

2. Project Objectives

The main objectives of this project were:

- To build a functional mechanical clock capable of displaying real time in a 7-segment format.
- To control 28 servo motors efficiently using an Arduino-based system.
- To use an RTC module for accurate timekeeping and time retention during power loss.
- To develop a stable power arrangement suitable for long-term wall-powered operation.
- To design and assemble a structured mechanical frame for mounting the servos and segments.
- To calibrate the segment positions for reliable motion and clear digit formation.
- To produce a complete working prototype that integrates electronics, software, and mechanical construction.

3. Main Components Used

The following components were used in the project:

- Arduino Uno
- DS1302 real-time clock module
- 2 × PCA9685 16-channel PWM servo driver boards
- 28 × SG90 micro servo motors
- Regulated 5V, 5A wall power supply
- Jumper wires and wiring harnesses
- 3D-printed frame parts and servo mounting parts
- Mechanical display segments and center dots
- Back support board

4. System Overview

The clock consists of four 7-segment mechanical digits representing hours and minutes, along with two dots in the center. Each segment is moved by a dedicated SG90 servo motor. The Arduino reads the current time from the DS1302 RTC module, converts the time into four separate digits, and sends PWM commands to the two PCA9685 driver boards. These driver boards then move the servos into their calibrated ON or OFF positions to form the correct numbers.

The two PCA9685 boards are used because the Arduino Uno alone does not have enough PWM outputs to control 28 servos directly. One board controls the two hour digits, while the second board controls the two minute digits. The RTC module ensures that the system always knows the current time, and its backup battery keeps the internal clock running even when the main supply is removed. The final structure is supported by a 3D-printed frame that holds the servo motors, segment supports, and display arrangement in a stable configuration.

5. Working Principle

Each of the four digits is formed by seven movable mechanical segments. A servo motor rotates each segment between two positions:

- an ON position, where the segment is visible from the front
- an OFF position, where the segment rotates away and becomes hidden

The Arduino continuously reads the time from the DS1302 RTC module and separates it into:

- hour tens
- hour units
- minute tens
- minute units

A lookup table in the program defines which of the seven segments must be ON or OFF for each digit from 0 to 9. Based on this table, the Arduino commands the PCA9685 boards to move the correct servos. Since the center segment can interfere with neighboring segments during movement, the software updates these segments in a controlled sequence to reduce the risk of collision.

6. Hardware Description

6.1 Arduino Uno

The Arduino Uno acts as the main controller of the system. It reads time from the RTC module, processes the display logic, and sends commands to the servo driver boards.

6.2 DS1302 RTC Module

The DS1302 real-time clock module provides accurate timekeeping for the clock. It communicates with the Arduino through dedicated digital pins. The module also includes a backup battery, which allows it to continue tracking time even when the main power supply is disconnected. This means that if the wall power is removed and later restored, the clock will still display the correct current time rather than resetting or stopping at the previously shown value.

6.3 PCA9685 PWM Driver Boards

The two PCA9685 boards generate PWM signals for the servo motors and greatly reduce the number of Arduino pins required. They communicate with the Arduino through the I2C interface using the SDA and SCL lines.

6.4 SG90 Servo Motors

A total of 28 SG90 micro servos are used, one for each segment. These convert the electrical control signals into mechanical movement.

6.5 Power Supply

The project uses a regulated 5V, 5A wall power supply. This allows the clock to operate continuously by simply plugging it into a standard household electrical outlet. This power approach is more practical than battery-based powering for permanent display use.

6.6 Mechanical Structure

The project includes a full 3D-printed frame with dedicated mounting positions for the servos, frame sections, central support parts, and segment covers. This improves stability, alignment, and overall appearance compared to loose mounting on a simple board.

7. Circuit and Wiring Description

The system contains three main electrical sections: timekeeping, control, and actuation.

The Arduino Uno is connected to the DS1302 RTC module using three digital pins:

- CLK to pin 6
- DAT to pin 7
- RST to pin 8

The PCA9685 servo driver boards are connected to the Arduino through I2C:

- SDA to A4
- SCL to A5

Only one driver board is directly connected to the Arduino, while the second board is chained to the first and addressed separately over I2C. The servo motors are powered from the regulated 5V supply through the driver boards. The same regulated supply is also used to power the Arduino. Wiring was routed mainly to the rear side of the display board in order to keep the front appearance clean and organized.

8. Software Description

The code is responsible for reading time, converting it into digit patterns, and moving the servos accordingly. It uses two main libraries:

- `virtuabotixRTC.h` for the DS1302 RTC module
- `Adafruit_PWMServoDriver.h` for the PCA9685 boards

The program defines calibrated PWM values for each servo's ON and OFF positions. Separate arrays are stored for the hour servos and minute servos because each servo may require slightly different calibration. The program also includes a digit lookup table for numbers 0 to 9.

When the minute changes, the display update function is called. This function first handles the middle segments carefully so that neighboring segments can move slightly away before the middle segment rotates. This reduces interference between adjacent moving parts. After that, the remaining segments are updated to form the correct time.

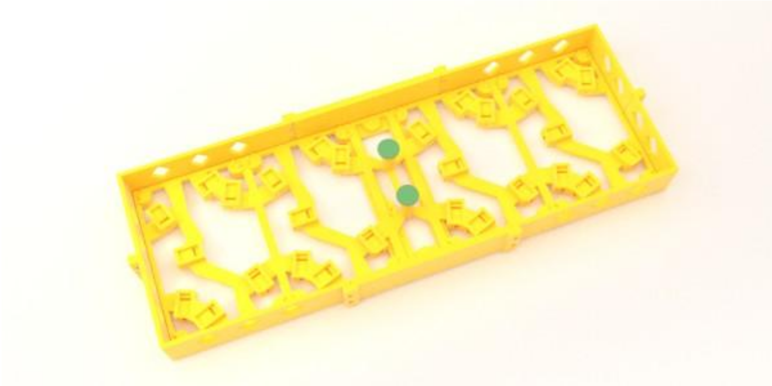
Before final installation of the servo arms, the system is powered and the code is uploaded so that all servos move into known positions. This makes alignment easier and reduces the chance of collisions during the first run.

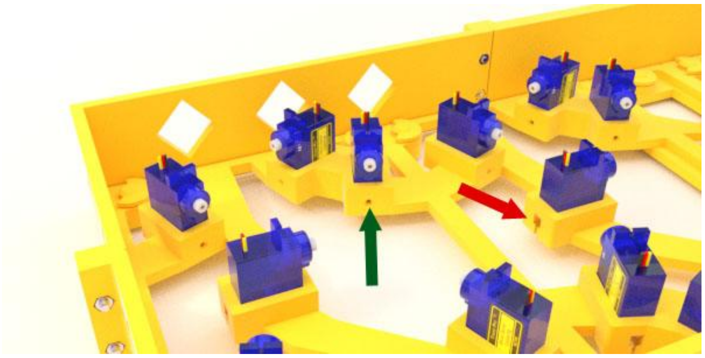
9. Mechanical Design and Assembly

The mechanical design consists of the display segments, servo bases, support spacers, center dots, and a full frame structure. Each printed segment is attached to a servo arm, and the servos are mounted into their designated slots in the structure. Small spacer blocks help keep each servo upright and aligned.

The display segments were prepared so that the front faces remain visible while the rear and side surfaces are darkened. This reduces the visibility of the segments when they rotate away from the viewer. The center dots are mounted between the hour and minute digits and aligned to the same visual level.

The frame was assembled first, followed by installation of the servo bases and center separator. After that, the servos were inserted into their mounting positions and their wires were routed toward the back of the board. Once calibration was completed, the segments were glued to the servo arms and arranged into the four-digit layout. The control boards and RTC module were then mounted on the rear side of the assembly.





10. Final Implementation Features

The final clock includes the following key features:

- Real-time mechanical hour and minute display
- 28 servo-actuated display segments
- Accurate timekeeping through the DS1302 RTC
- RTC backup battery for time retention during power loss
- Efficient multi-servo control using two PCA9685 driver boards
- Direct wall-powered operation through a regulated 5V, 5A supply
- Organized 3D-printed frame and servo mounting structure
- Rear wiring layout for a cleaner front appearance

11. Challenges Faced

Several practical challenges were encountered during the project:

- Calibrating 28 servos accurately so that all segments align properly
- Preventing adjacent segments from touching during movement
- Managing and organizing a large number of servo wires
- Ensuring that the power supply was stable enough for many servos
- Aligning the servo arms and segments correctly before final gluing
- Achieving a clean and stable mechanical layout for the complete display

These challenges required careful testing, calibration, and assembly to achieve reliable operation.

12. Conclusion

The mechanical 7-segment display clock project successfully demonstrates the integration of Arduino programming, real-time clock interfacing, servo control, PWM expansion, power distribution, and 3D-printed mechanical construction into one complete system. The final prototype displays the correct current time using four large mechanical digits and two center dots, driven by 28 individually controlled servo motors.

The use of the DS1302 RTC module with its backup battery ensures that the system retains accurate time even when main power is disconnected. The regulated wall-powered 5V supply makes the clock practical for continuous everyday use, while the 3D-printed frame improves structural stability and presentation. Overall, the project provided strong practical experience in embedded systems, electromechanical actuation, wiring, calibration, and full system integration.

13. Future Improvements

Possible future improvements for the project include:

- adding a seconds display
- designing a custom PCB for cleaner wiring
- improving the frame aesthetics further
- using quieter or higher-precision servos
- adding a brightness or night mode concept using external lighting
- integrating automatic time adjustment from a more advanced RTC or network source